Звіт

до лабораторної роботи 2

з навчальної дисципліни

«Актуальні проблеми "Data Mining"»

Тема: Пошук асоціативних правил, кластеризація

Виконав: студент групи ММШІ-1 Красіков Антон Ігорович

1. Опис та постановка задачі

Метою даної роботи є програмна реалізація, дослідження та порівняльний аналіз базових алгоритмів навчання без вчителя. Робота структурно поділяється на два незалежних напрямки:

1. **Пошук асоціативних правил:** Реалізувати алгоритми Apriori та FP-Growth для знаходження неочевидних закономірностей у транзакційних даних. Порівняти власні імплементації з оптимізованими бібліотечними рішеннями за критерієм швидкодії.
2. **Кластеризація просторових даних:** Реалізувати спектр алгоритмів кластеризації: центроїдні (k-Means, k-Medians), ієрархічні (агломеративна з одиночним зв'язком / алгоритм найближчого сусіда) та плотносні (DBSCAN). Дослідити їхню поведінку на наборах даних із різною топологією та наявністю аномалій (викидів).

2. Теоретичні відомості

2.1. Пошук асоціативних правил

Задача полягає у знаходженні правил вигляду $A \rightarrow B$ на основі трьох ключових метрик:

- **Support (підтримка):** Частота зустрічальності набору товарів у всій базі транзакцій.
- **Confidence (достовірність):** Умовна ймовірність того, що при наявності A в чеку з'явиться і B.
- **Lift (підйом):** Відношення спостережуваної достовірності до очікуваної (припущення про незалежність товарів). Оцінює реальну силу зв'язку.

Досліджувані алгоритми:

- **Apriori:** Працює за принципом генерації кандидатів. Використовує властивість антимонотонності (якщо набір не є частим, то всі його надмножини також не часті). Є ресурсоємним ($O(2^N)$ у найгіршому випадку), оскільки вимагає багаторазового сканування бази даних.
- **FP-Growth:** Оптимізований алгоритм, який відмовляється від генерації кандидатів. Натомість він стискає базу транзакцій у спеціальну структуру даних в оперативній пам'яті (FP-Tree - префіксне дерево) та рекурсивно видобуває часті набори, що забезпечує значний приріст швидкодії.

2.2. Алгоритми кластеризації

- **k-Means та k-Medians (центроїдні алгоритми):** Ітеративні алгоритми (EM-подібні), що мінімізують внутрішньокластерну відстань. k-Means використовує Евклідову відстань та середнє арифметичне (чутливий до викидів). k-Medians використовує Манхеттенську відстань та медіану, що робить його більш стійким до аномалій.
- **Ієрархічна кластеризація:** Жадібний "висхідний" алгоритм. На кожному кроці знаходить дві найближчі точки (або кластери) і об'єднує їх. Алгоритм "найближчого сусіда" дозволяє знаходити кластери складної нелінійної форми, але має високу обчислювальну складність $O(N^3)$ при наївній реалізації.
- **DBSCAN:** Групує точки, що щільно розташовані разом (мають щонайменше min_samples сусідів у радіусі eps), і позначає точки в регіонах з низькою щільністю як шум (викиди). Здатен знаходити кластери довільної форми та не потребує попереднього задання кількості кластерів.

3. Опис наборів даних

Для коректного тестування специфічних властивостей алгоритмів було використано три різні набори даних:

1. **Groceries Dataset (транзакційні дані):** Набір реальних чеків із супермаркету. Дані представлені у вигляді списків змінної довжини. Використовувався для знаходження асоціативних правил. Для сумісності з бібліотечними функціями (mlxtend) дані додатково трансформувались за допомогою One-Hot Encoding.

2. **Blobs з викидами (синтетичний):** Згенерований набір із трьох чітких сферичних кластерів (300 точок), до якого штучно додано 5 екстремальних точок-викидів. Мета: продемонструвати вразливість k-Means та стійкість до них k-Medians.

3. **Moons (синтетичний):** Згенерований набір нелінійних даних у формі двох вкладених дуг із невеликим рівнем шуму (300 точок). Мета: продемонструвати математичні обмеження центроїдних алгоритмів та переваги DBSCAN та ієрархічної кластеризації.

4. Хід роботи

4.1. Архітектура пошуку асоціативних правил

- Було програмно реалізовано алгоритм Apriori з ручною генерацією кандидатів та фільтрацією за порогом підтримки.
- Процес пошуку був архітектурно розділений на два ізольовані етапи: генерація частих наборів та генерація математичних правил на основі метрик confidence та lift. Це дозволило ефективно підбирати пороги без перерахунку всієї бази.
- Як альтернативу Apriori, було також програмно реалізовано алгоритм FP-Growth. Для цього спроектовано власну об'єктно-орієнтовану структуру даних - префіксне дерево (FP-Tree) з таблицею заголовків та системою горизонтальних зв'язків між вузлами. Реалізовано рекурсивний алгоритм побудови умовних дерев та отримання з них частих наборів, що дозволило алгоритму працювати без кроку генерації кандидатів.
- Написано функціонал для бенчмаркінгу власних реалізацій у порівнянні з бібліотекою mlxtend при поступовому зменшенні параметра min_support.

4.2. Архітектура кластерного аналізу

Систему кластеризації було побудовано з використанням принципів об'єктно-орієнтованого програмування:

- Створено абстрактний базовий клас `BaseClusterer` із жорстким контрактом методів.
- Для алгоритмів сімейства **K** реалізовано загальний абстрактний клас, що керує циклом очікування-максимізації (EM-loop). Впроваджено

інженерний захист від "порожніх кластерів" (автоматична реініціалізація центроїда, якщо він втрачає всі точки). Класи KMeans та KMedians реалізують лише специфічні математичні операції (визначення дистанції та центру). Усі обчислення векторизовані через матричні операції NumPy.

- При реалізації ієрархічної кластеризації наївний пошук по масивах було замінено на O(1) словник прямого доступу (point-to-cluster mapping), що суттєво зменшило час виконання.
- DBSCAN реалізовано за допомогою обходу графа в ширину (BFS) для ефективного розширення "ядер" кластерів.

5. Оцінка результатів та висновки

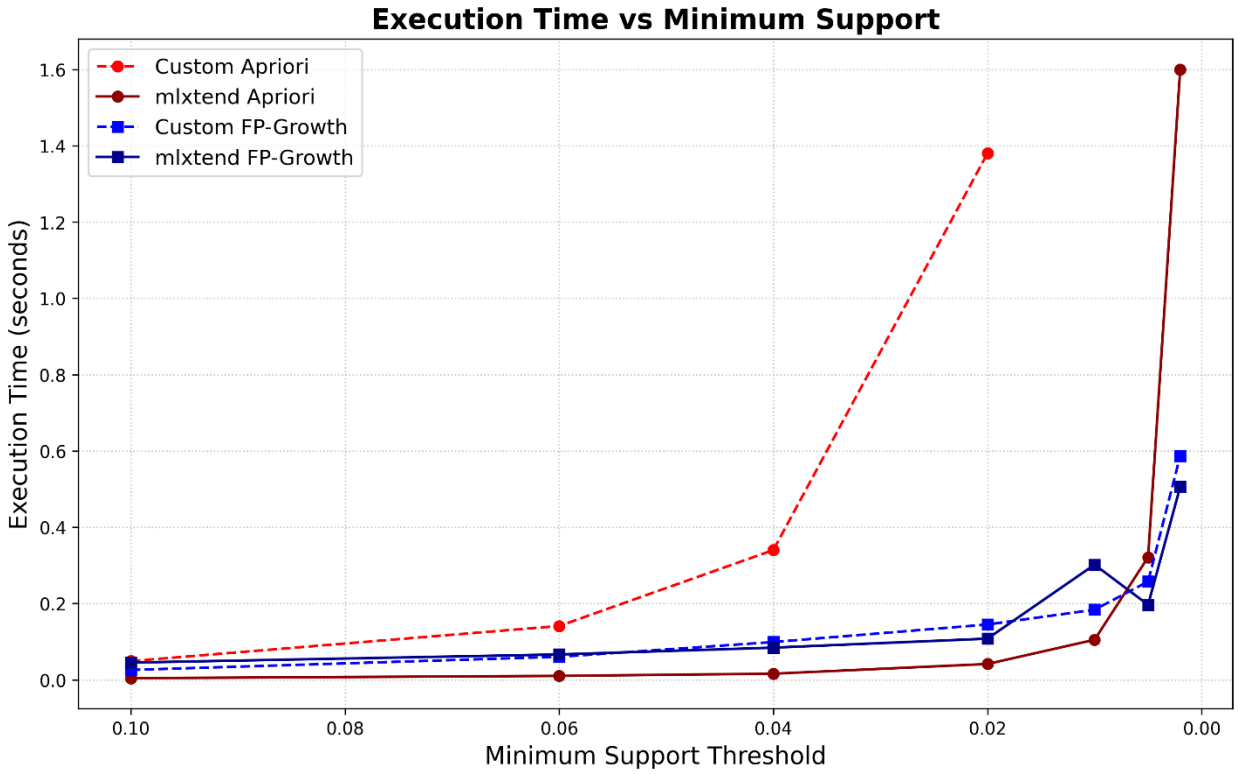
5.1. Аналіз асоціативних правил

Власні реалізації обох алгоритмів (Apriori та FP-Growth) успішно знайшли абсолютно ідентичні набори правил, які повністю збігаються з результатами еталонної бібліотеки mlxtend за всіма ключовими метриками (support, confidence, lift). Це підтверджує математичну коректність написаного коду.

```
====== APRIORI ==========================================================================

                                  Custom   Library (mlxtend)
Frequent itemsets found               31                 31
Association rules found                6                  6
Execution time (sec)              0.2654             0.0167

Top-5 rules by Lift  [Custom]:
Antecedents                          Consequents                Support     Confidence    Lift
-----------------------------------  --------------------------  ----------  ------------  --------
yogurt                               whole milk                 0.0560      0.4016        1.5717
whole milk                           yogurt                     0.0560      0.2193        1.5717
other vegetables                     whole milk                 0.0748      0.3868        1.5136
whole milk                           other vegetables           0.0748      0.2929        1.5136
whole milk                           rolls/buns                 0.0566      0.2216        1.2050

Top-5 rules by Lift  [Library (mlxtend)]:
Antecedents                          Consequents                Support     Confidence    Lift
-----------------------------------  --------------------------  ----------  ------------  --------
yogurt                               whole milk                 0.0560      0.4016        1.5717
whole milk                           yogurt                     0.0560      0.2193        1.5717
other vegetables                     whole milk                 0.0748      0.3868        1.5136
whole milk                           other vegetables           0.0748      0.2929        1.5136
whole milk                           rolls/buns                 0.0566      0.2216        1.2050


====== FP-GROWTH ========================================================================

                                  Custom   Library (mlxtend)
Frequent itemsets found               31                 31
Association rules found                6                  6
Execution time (sec)              0.0849             0.0742

Top-5 rules by Lift  [Custom]:
Antecedents                          Consequents                Support     Confidence    Lift
-----------------------------------  --------------------------  ----------  ------------  --------
yogurt                               whole milk                 0.0560      0.4016        1.5717
whole milk                           yogurt                     0.0560      0.2193        1.5717
other vegetables                     whole milk                 0.0748      0.3868        1.5136
whole milk                           other vegetables           0.0748      0.2929        1.5136
whole milk                           rolls/buns                 0.0566      0.2216        1.2050

Top-5 rules by Lift  [Library (mlxtend)]:
Antecedents                          Consequents                Support     Confidence    Lift
-----------------------------------  --------------------------  ----------  ------------  --------
yogurt                               whole milk                 0.0560      0.4016        1.5717
whole milk                           yogurt                     0.0560      0.2193        1.5717
other vegetables                     whole milk                 0.0748      0.3868        1.5136
whole milk                           other vegetables           0.0748      0.2929        1.5136
rolls/buns                           whole milk                 0.0566      0.3079        1.2050
```

Аналіз швидкодії та обчислювальної складності:

Зменшення порогу min_support призводить до експоненційного зростання кількості кандидатів.



На основі побудованих графіків залежності часу виконання від порогу min_support виявлено наступні закономірності:

1. **Перевага структури даних:** Власна реалізація FP-Growth продемонструвала беззаперечну перевагу у швидкості над власним та бібліотечним Apriori, особливо при низьких значеннях підтримки. Завдяки стисненню транзакцій у FP-Tree, алгоритм успішно уникнув "комбінаторного вибуху", від якого страждає Apriori під час генерації кандидатів.

2. **Проблема мови програмування:** Незважаючи на алгоритмічну перевагу FP-Growth над Apriori, порівняння власної реалізації з бібліотечною версією mlxtend FP-Growth показало незначне відставання кастомного коду у швидкості. Це очікуваний інженерний результат: рекурсивний обхід складних структур на чистому Python створює великий накладний час. Бібліотечні ж рішення використовують оптимізовані C-розширення.

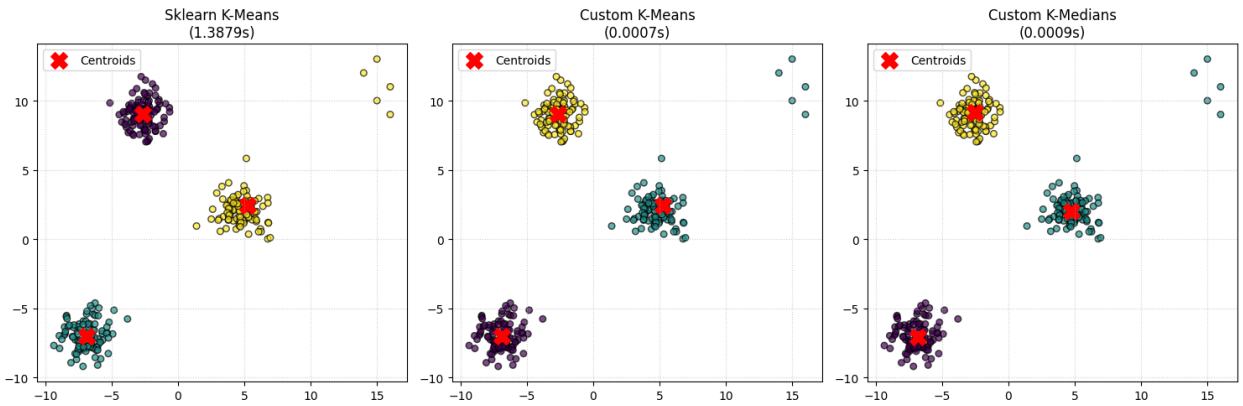
Висновок: Для аналізу транзакційних даних алгоритм FP-Growth є значно ефективнішим за Apriori завдяки кращій асимптотичній складності. Проте експеримент доводить, що для ресурсоємних задач классу Data Mining вибір низькорівневої мови програмування (або оптимізованих бібліотек) відіграє мало не таку ж критичну роль, як і вибір самого алгоритму.

5.2. Аналіз алгоритмів кластеризації

Тестування на різних топологіях даних виявило фундаментальні відмінності в математичних моделях алгоритмів.

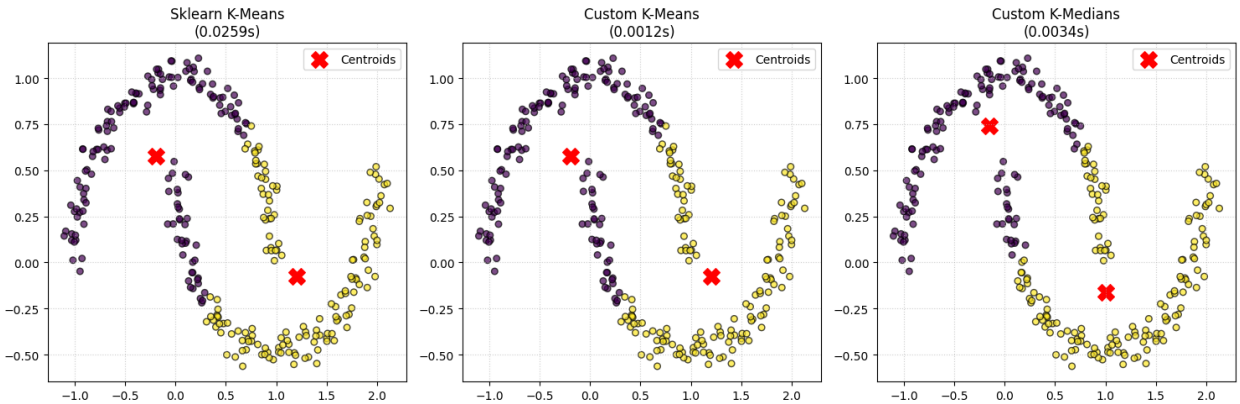
Експеримент 1: Вплив викидів (blobs + outliers)

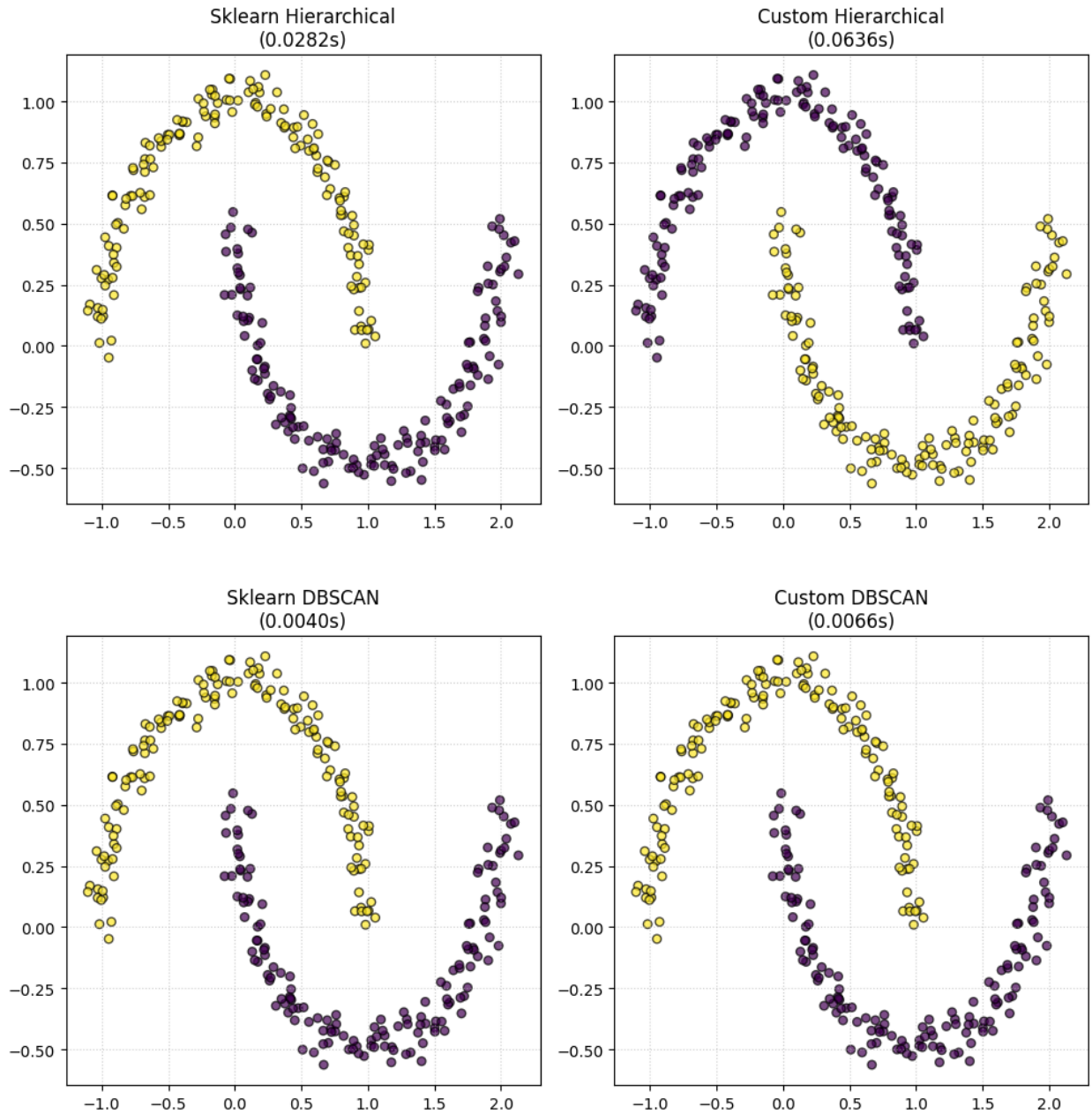
- k-Means: Центроїд одного з кластерів змістився у бік аномальних точок, спотворивши межі кластера (наслідок чутливості середнього арифметичного до квадратів відстаней).
- k-Medians: Повністю проігнорував аномалії. Центроїди залишилися в геометричних центрах щільних скупчень точок.



Експеримент 2: Нелінійна топологія (Moons)

- k-Means / k-Medians: Зазнали поразки. Центроїдні алгоритми намагаються розділити простір лінійними гіперплощинами (утворюючи комірки Вороного), тому вони буквально "розрізали" напівмісяці навпіл.
- Ієрархічна (найближчий сусід): Успішно виділила обидва напівмісяці, оскільки відстань вимірювалась між найближчими крайовими точками, дозволяючи кластеру "розтікатися" вздовж дуги.
- DBSCAN: Так само ідеально розпізнав складну форму розподілу даних.



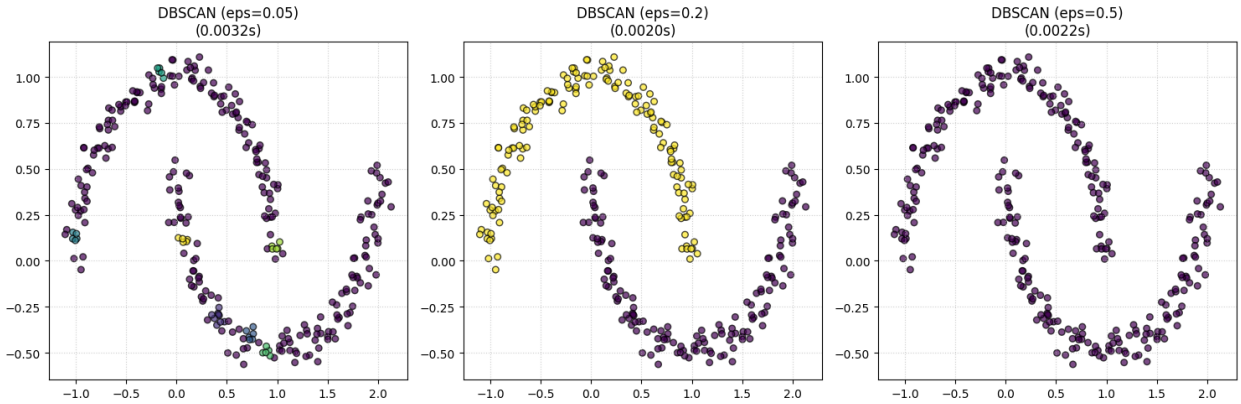


Експеримент 3: Чутливість гіперпараметрів DBSCAN

Було доведено критичну важливість параметра eps (радіус пошуку).

- При eps = 0.05 алгоритм не зміг знайти зв'язки і роздробив дані на мікро-кластери та шум.
- При eps = 0.2 було досягнуто ідеального розділення.

- При eps = 0.5 радіус перевищив відстань між двома напівмісяцями, і алгоритм злив їх у єдиний монолітний кластер.



Загальний висновок:

Універсального алгоритму кластеризації не існує. Вибір методу диктується природою даних: для сферичних бізнес-сегментів чудово підходять k-Means/k-Medians, тоді як для просторових даних зі складною геометрією та зашумленістю необхідно використовувати алгоритми по типу DBSCAN.

Посилання на дані транзакцій:

`https://www.kaggle.com/datasets/irfanasrullah/groceries`